

SUPPORT PLATFORM

Chatbot IA multi-tenant – WAGAN AI

Documentation Architecture

Easy Event Support Platform

Une architecture unique pour servir plusieurs projets

Réutilisabilité – Isolation des données – Scalabilité – Personnalisation

Document réalisé par : Fatou Fall

Statut : Étudiante en double cursus Master 2
Génie Logiciel
Sciences des Données et Applications
(Statistique, Économétrie et Modélisation)

Établissement : Université Iba Der Thiam de Thiès

Date : 27 Juillet 2026

Version : 1.0

Contents

1	Présentation générale	2
1.1	Qu'est-ce que la Support Platform ?	2
1.2	Objectifs	2
2	Architecture multi-tenant	2
2.1	Définition du multi-tenant	2
2.2	Les acteurs du multi-tenant	2
3	Preuves dans le code	2
3.1	Modèle Tenant – la base du multi-tenant	3
3.2	Données liées au Tenant	3
3.3	API – récupération du Tenant par clé API	3
3.4	Moteur IA – utilisation du Tenant pour les réponses	4
3.5	Admin – filtrage par Tenant	4
4	Diagramme d'architecture	5
4.1	Vue d'ensemble	5
4.2	Flux de données multi-tenant	6
5	Flux de données	6
5.1	Requête d'un client	6
5.2	Traitement dans l'API	6
5.3	Moteur IA avec Tenant	7
6	Cas pratique : intégration de Easy Planning	7
6.1	Étape 1 : ajouter le composant chatbot dans Easy Planning	7
6.2	Étape 2 : configurer le Tenant dans l'admin Django	8
6.3	Étape 3 : vérifier le filtrage par Tenant dans l'admin	9
6.4	Étape 4 : vérifier le fonctionnement	9
6.5	Étape 5 : résumé des fichiers modifiés	10
7	Conclusion	10
7.1	Résumé des preuves	10
7.2	Avantages du multi-tenant	10
7.3	Architecture multi-tenant – synthèse	10
7.4	Exemple concret : intégration de Easy Planning	11

1 Présentation générale

1.1 Qu'est-ce que la Support Platform ?

La **Support Platform** est une solution de chatbot IA conçue pour être intégrable à n'importe quel projet. Elle repose sur une architecture **multi-tenant** qui permet à une seule instance de servir plusieurs applications clientes de manière isolée et sécurisée.

1.2 Objectifs

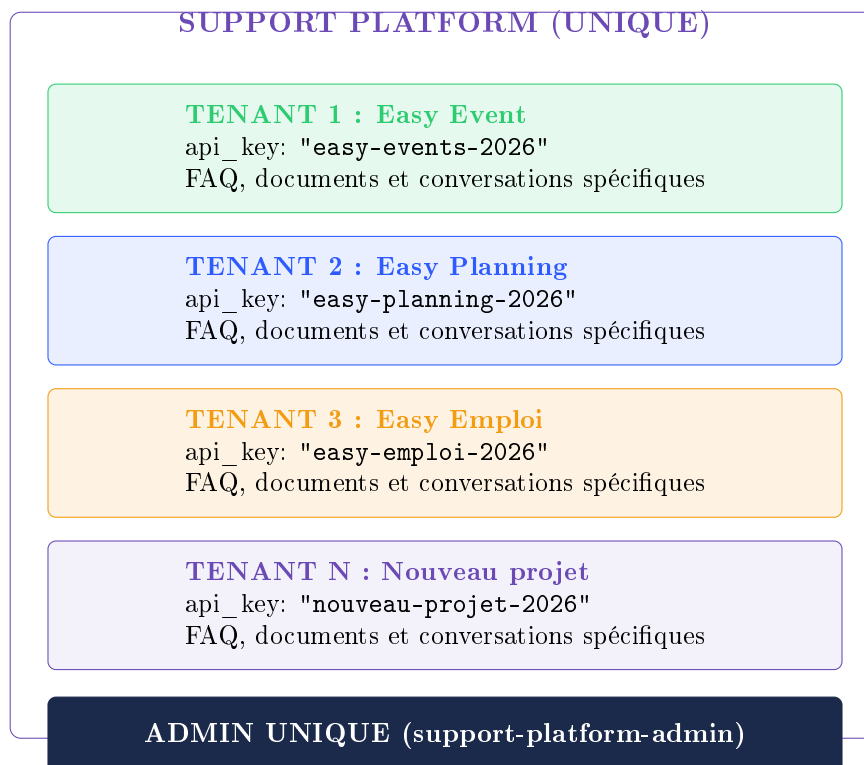
Objectif	Description
Réutilisabilité	Un seul backend pour plusieurs projets
Isolation des données	Chaque projet a ses propres FAQ, documents et conversations
Scalabilité	Ajouter de nouveaux projets sans déployer de nouveaux serveurs
Personnalisation	Chaque projet peut avoir son propre branding et ses propres réponses

2 Architecture multi-tenant

2.1 Définition du multi-tenant

Le **multi-tenant** est un modèle d'architecture où une seule instance d'application sert plusieurs clients (*tenants*). Chaque tenant a ses propres données isolées, mais partage la même infrastructure.

2.2 Les acteurs du multi-tenant



3 Preuves dans le code

3.1 Modèle Tenant – la base du multi-tenant

Fichier : chatbot/models.py

```
class Tenant(models.Model):
    """Application cliente - Chaque projet est un Tenant"""
    name = models.CharField(max_length=100)
    api_key = models.CharField(max_length=100, unique=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

Preuve

Chaque projet est représenté par un `Tenant` avec un `api_key` unique.

3.2 Données liées au Tenant

Fichier : chatbot/models.py

```
class KnowledgeItem(models.Model):
    """Base de connaissance - FAQ spécifiques a chaque projet"""
    tenant = models.ForeignKey(Tenant, on_delete=models.CASCADE,
                              related_name='knowledge_items')
    question = models.CharField(max_length=500, blank=True, null=True)
    answer = models.TextField()
    category = models.CharField(max_length=100, blank=True, default='general')
    created_at = models.DateTimeField(auto_now_add=True)

class Document(models.Model):
    """Documents uploadés - Spécifiques a chaque projet"""
    tenant = models.ForeignKey(Tenant, on_delete=models.CASCADE,
                              related_name='documents')
    title = models.CharField(max_length=255)
    file = models.FileField(upload_to='documents/', blank=True, null=True)
    document_type = models.CharField(max_length=10, choices=DOCUMENT_TYPES, default='pdf')
    content = models.TextField(blank=True)
    uploaded_at = models.DateTimeField(auto_now_add=True)

class Conversation(models.Model):
    """Conversations - Isolées par projet"""
    user = models.ForeignKey(User, on_delete=models.CASCADE,
                              related_name='conversations', null=True, blank=True)
    tenant = models.ForeignKey(Tenant, on_delete=models.CASCADE,
                              related_name='conversations')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    escalated = models.BooleanField(default=False)
```

Preuve

Chaque `KnowledgeItem`, `Document` et `Conversation` est lié à un `Tenant` spécifique via `ForeignKey`. Les données sont donc isolées par projet.

3.3 API – récupération du Tenant par clé API

Fichier : chatbot/views.py

```

class SendMessageView(APIView):
    def post(self, request):
        # 1. Recuperation de l'api_key depuis la requete
        data = serializer.validated_data
        api_key = data['api_key']

        # 2. Recherche du tenant correspondant
        try:
            tenant = Tenant.objects.get(api_key=api_key) # UNIQUE TENANT
        except Tenant.DoesNotExist:
            return Response({"error": "Cle API invalide"},
                            status=status.HTTP_401_UNAUTHORIZED)

        # 3. Creation de la conversation liee au tenant
        conversation = Conversation.objects.create(tenant=tenant, user=django_user)

        # 4. Appel du moteur IA avec le tenant
        bot_response_text, bot_success = get_bot_response(content, tenant,
                                                            sanctum_token)
        # Le tenant est passe au moteur IA

```

Preuve

L'API utilise la clé API pour identifier le Tenant et isole les données par projet.

3.4 Moteur IA – utilisation du Tenant pour les réponses

Fichier : chatbot/ai_engine.py

```

def mock_response(message_content, tenant, user_data=None, ...):
    """Reponses adaptees au tenant (projet)"""

    # 1. RECHERCHE DANS LA BASE DE CONNAISSANCE DU TENANT
    if hasattr(tenant, 'knowledge_items'):
        for item in tenant.knowledge_items.all(): # FAQ specifiques au tenant
            if item.question and item.question.lower() in message_lower:
                return item.answer, True

    # 2. RECHERCHE DANS LES DOCUMENTS DU TENANT
    if hasattr(tenant, 'documents'):
        for doc in tenant.documents.all(): # Documents specifiques au tenant
            if doc.content and len(message_lower) > 5:
                if doc.content.lower().find(message_lower[:20]) != -1:
                    return f"D'apres '{doc.title}' :\n\n{doc.content[:500]}...", True

```

Preuve

Le moteur IA interroge uniquement les knowledge_items et documents du tenant passé en paramètre.

3.5 Admin – filtrage par Tenant

Fichier : support-platform-admin/src/App.jsx

```

// Le dashboard supporte le filtrage par tenant
const [selectedTenant, setSelectedTenant] = useState('all');

// Filtrer les conversations par tenant

```

```
const filteredConversations = conversations.filter(conv => {
  if (selectedTenant === 'all') return true;
  return conv.tenant_id === parseInt(selectedTenant);
});

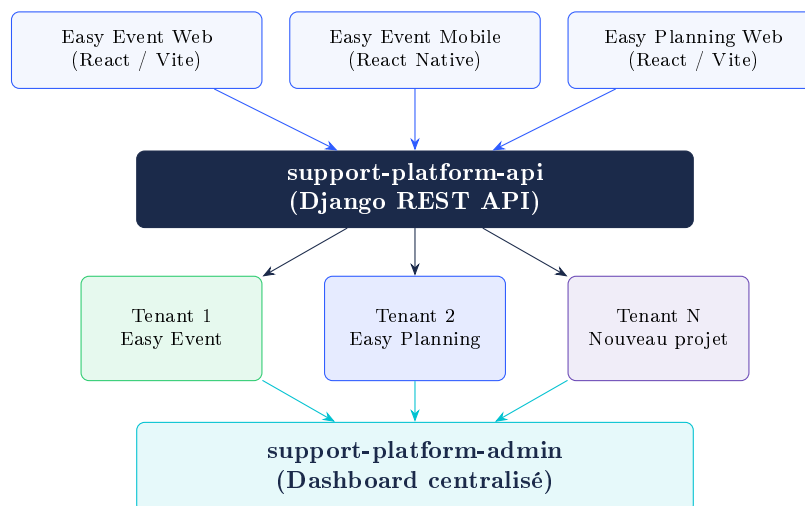
// Affichage du selecteur de tenant
<select onChange={e => setSelectedTenant(e.target.value)}>
  <option value="all">Tous les projets</option>
  <option value="1">Easy Event</option>
  <option value="2">Easy Planning</option>
  <option value="3">Easy Emploi</option>
</select>
```

Preuve

L'admin peut filtrer les conversations par `tenant_id`.

4 Diagramme d'architecture

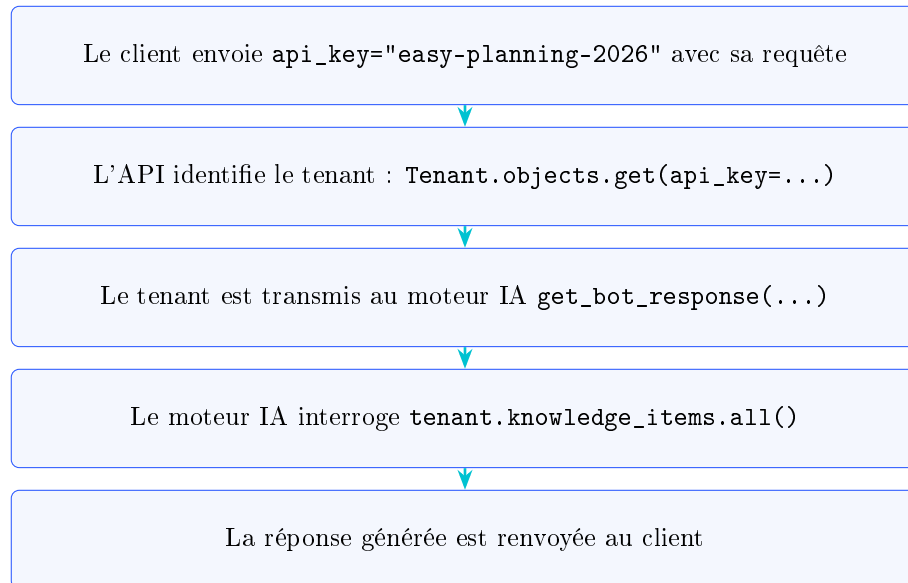
4.1 Vue d'ensemble



Note

Chaque requête client transporte un `api_key` identifiant son tenant. L'API centralise la logique métier tandis que l'admin agrège et filtre les conversations projet par projet.

4.2 Flux de données multi-tenant



5 Flux de données

5.1 Requête d'un client

```
POST /api/message/
Headers:
  Authorization: Bearer <sanctum_token>

Body:
{
  "api_key": "easy-planning-2026",
  "content": "comment creer un planning",
  "conversation_id": null
}
```

5.2 Traitement dans l'API

```
# 1. Verification de l'api_key
tenant = Tenant.objects.get(api_key="easy-planning-2026")
# -> tenant.name = "Easy Planning"

# 2. Creation de la conversation
conversation = Conversation.objects.create(tenant=tenant, user=user)
# -> conversation.tenant = tenant (Easy Planning)

# 3. Appel du moteur IA
bot_response = get_bot_response(content, tenant, sanctum_token)
# -> Le moteur IA utilise tenant.knowledge_items

# 4. Sauvegarde du message
Message.objects.create(
    conversation=conversation,
    role='bot',
    content=bot_response
)
```

5.3 Moteur IA avec Tenant

```
def mock_response(message_content, tenant, ...):
    # Recherche dans la base de connaissance du tenant
    for item in tenant.knowledge_items.all():
        if item.question in message_content:
            return item.answer

    # Recherche dans les documents du tenant
    for doc in tenant.documents.all():
        if doc.content.find(message_content) != -1:
            return doc.content

    # Si rien n'est trouve -> escalade
    return None, False
```

6 Cas pratique : intégration de Easy Planning

Scénario

Un développeur t'invite sur le dépôt GitHub de **Easy Planning** et te demande d'intégrer le chatbot **WAGAN AI**. L'admin doit voir uniquement les escalades de Easy Planning dans le dashboard.

6.1 Étape 1 : ajouter le composant chatbot dans Easy Planning

1. Créer le fichier ChatbotWidget.jsx

Emplacement : easy-planning/src/components/ChatbotWidget.jsx

```
// easy-planning/src/components/ChatbotWidget.jsx
import React, { useState, useEffect, useRef } from 'react';
import axios from 'axios';

// Configuration spécifique à Easy Planning
const API_BASE = 'https://support-platform-api-0h06.onrender.com';
const API_KEY = 'easy-planning-2026'; // Cle API pour Easy Planning
const TENANT_ID = 2; // ID du tenant Easy Planning
const APP_NAME = 'Easy Planning';

// Avatar de l'agent
const AGENT_AVATAR = '/images/wagan-ai-avatar.jpg';

// ... (le reste du composant ChatbotWidget identique)
// Mais avec les valeurs spécifiques ci-dessus
```

Points importants

API_KEY = 'easy-planning-2026' (unique pour ce projet) ; TENANT_ID = 2 (identifiant du tenant en base de données) ; APP_NAME = 'Easy Planning' (affiché dans le header).

2. Importer et utiliser le composant dans App.jsx

Emplacement : easy-planning/src/App.jsx

```
// easy-planning/src/App.jsx
import React from 'react';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
```

```
import ChatbotWidget from './components/ChatbotWidget';

function App() {
  return (
    <Router>
      <Routes>
        { /* Vos routes existantes */ }
        <Route path="/" element={<Home />} />
        <Route path="/planning" element={<Planning />} />
      </Routes>

      { /* Chatbot visible sur toutes les pages */ }
      <ChatbotWidget />
    </Router>
  );
}

export default App;
```

3. Ajouter l'image de l'avatar

Emplacement : easy-planning/public/images/wagan-ai-avatar.jpg

```
# Copier l'image depuis Easy Event
cp easy-event-web/src/assets/images/images.jpg \
  easy-planning/public/images/wagan-ai-avatar.jpg
```

6.2 Étape 2 : configurer le Tenant dans l'admin Django

1. Accéder à l'admin Django

URL : <https://support-platform-api-0h06.onrender.com/admin>

2. Créer le Tenant "Easy Planning"

```
# Dans Django shell
from chatbot.models import Tenant

tenant = Tenant.objects.create(
    name="Easy Planning",
    api_key="easy-planning-2026"
)
print(f"Tenant cree : {tenant.name} (ID: {tenant.id})")
# -> Tenant cree : Easy Planning (ID: 2)
```

3. Ajouter des FAQ spécifiques à Easy Planning

Question	Réponse
"c'est quoi easy planning"	"Easy Planning est une application de gestion de plannings..."
"comment créer un planning"	"Pour créer un planning, cliquez sur 'Nouveau planning'..."
"comment ajouter une tâche"	"Pour ajouter une tâche, utilisez le bouton '+'..."

```
# Dans Django shell
from chatbot.models import KnowledgeItem
```

```
KnowledgeItem.objects.create(
    tenant=tenant,
    question="comment creer un planning",
    answer="Pour creer un planning, cliquez sur le bouton 'Nouveau planning'..."
)

KnowledgeItem.objects.create(
    tenant=tenant,
    question="c'est quoi easy planning",
    answer="Easy Planning est une application de gestion de plannings..."
)
```

4. Ajouter des documents spécifiques

```
curl -X POST https://support-platform-api-0h06.onrender.com/api/documents/upload/ \
-F "api_key=easy-planning-2026" \
-F "title=Guide Easy Planning" \
-F "file=@guide-easy-planning.pdf"
```

6.3 Étape 3 : vérifier le filtrage par Tenant dans l'admin

```
// Le dashboard supporte le filtrage par tenant
const [selectedTenant, setSelectedTenant] = useState('all');

const filteredConversations = conversations.filter(conv => {
  if (selectedTenant === 'all') return true;
  return conv.tenant_id === parseInt(selectedTenant);
});
```

Résultat attendu

Quand l'admin sélectionne "Easy Planning", il voit uniquement les escalades de Easy Planning.

6.4 Étape 4 : vérifier le fonctionnement

```
# Dans Easy Planning
npm start

# Poser une question spécifique
"comment creer un planning"
# -> Le chatbot repond avec la FAQ configurée

# Poser une question non reconnue
"je ne comprends pas"
# -> La conversation est escaladée
```

6.5 Étape 5 : résumé des fichiers modifiés

Fichier	Modification
easy-planning/src/components/ChatbotWidget.jsx	Ajout du composant
easy-planning/src/App.jsx	Import et utilisation du composant
easy-planning/public/images/wagan-ai-avatar.jpg	Ajout de l'image
support-platform-admin	Déjà configuré (filtrage par tenant)
support-platform-api/admin	Déjà configuré (création du tenant)

7 Conclusion

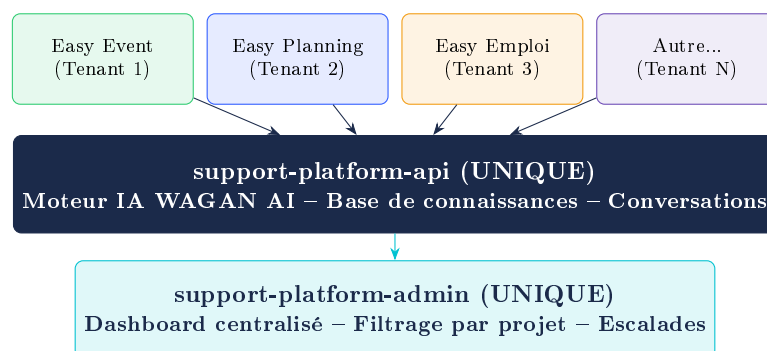
7.1 Résumé des preuves

Point	Preuve dans le code	Fichier
Modèle Tenant	<code>class Tenant(models.Model)</code>	models.py
Données liées au Tenant	<code>tenant = models.ForeignKey(Tenant)</code>	models.py
API utilise Tenant	<code>Tenant.objects.get(api_key=...)</code>	views.py
Moteur IA utilise Tenant	<code>tenant.knowledge_items.all()</code>	ai_engine.py
Admin gère les Tenants	<code>admin.site.register(Tenant)</code>	admin.py
Admin filtre par Tenant	<code>conv.tenant_id == selectedTenant</code>	App.jsx

7.2 Avantages du multi-tenant

Avantage	Description
Économie de ressources	Un seul serveur pour plusieurs projets
Maintenance simplifiée	Une seule base de code à maintenir
Isolation des données	Les données de chaque projet sont séparées
Scalabilité	Ajouter des projets sans déployer de nouveaux serveurs
Administration centralisée	Un seul dashboard pour tous les projets

7.3 Architecture multi-tenant – synthèse



UN SEUL SERVEUR → UN SEUL DASHBOARD → MULTIPLES PROJETS

Conclusion finale

La Support Platform est bien conçue pour être multi-tenant et peut être intégrée à n'importe quel projet.

7.4 Exemple concret : intégration de Easy Planning

Étape	Action	Résultat
1	Ajouter <code>ChatbotWidget.jsx</code> dans Easy Planning	Le chatbot apparaît dans l'application
2	Configurer <code>API_KEY = "easy-planning-2026"</code>	Le chatbot communique avec le bon tenant
3	Créer le Tenant "Easy Planning" dans l'admin	Le projet est reconnu par l'API
4	Ajouter des FAQ spécifiques	Le chatbot répond aux questions spécifiques
5	L'admin filtre par tenant	L'admin voit uniquement les escalades de Easy Planning

Document réalisé par : **Fatou Fall**

Étudiante en double cursus Master 2 – Génie Logiciel & Sciences des Données et Applications (Statistique, Économétrie et Modélisation)

Université Iba Der Thiam de Thiès

Date : 27 Juillet 2026 – Version : 1.0